KV 调度 Benchmark 数据集

任务窃取（A5）测试与 Chase-Lev 变体评估

数据集：KV_Scheduling_Benchmark_Student_20261104

课程项目“内存键值存储引擎中的轻量级任务调度优化”

2026 年 11 月 4 日

摘要

本文记录使用课程组提供的 KV 调度基准数据集（2026-11-04 版）对任务窃取（A5）进行的独立测试。测试分为三部分：先原样运行官方基准的全部自动化检查与主套件，确认数据与代码管线可用；再把 Chase-Lev 双端队列语义接入同一仿真引擎，作为 A5 的一个可比较变体；最后做阈值与探测成本敏感性实验，检查该变体在什么条件下值得采用。

主要结论如下。第一，把官方四种算法放在一起，P99 排名是 FIFO ≈ Priority > Steal ≫ Local：FIFO 与 Priority 在 13/18 个场景并列最低，Priority 在 4 个 T5 场景略优，Steal 只在 T4_K100 胜出，Local 从未胜出。但这个排名是在调度、探测、迁移成本全为 0、全局队列没有锁竞争的模型下得到的，不能直接外推。第二，官方 A5（最长队列受害者、队尾窃取）相对 A4 本地队列显著降低前台 P99：按 T1—T5 组均值，A5 的 P99 只有 A4 的 5.7%—24.8%，即降低 75%—94%。第三，同键热点 D1 中四种算法 P99 都约为 11 202，其中键依赖等待约 11 201，说明窃取无法突破同键串行上限，这与设计文档的预期一致。第四，Chase-Lev 变体在阈值 k=0、零探测成本时与最长队列策略基本持平（12/18 个场景不劣于或优于，平均 P99 比 0.983）；当每次探测计入 0.05 单位成本、且失败探测也计费时，只有 3/18 个场景优于最长队列（平均 1.84，最好 0.632、最差 2.42），不能再说“随机早停全面胜出”。

第五，也是最重要的修正：把阈值设为 k=2 在 8 worker、70% 负载下会造成严重退化，P99 达到最长队列策略的 2.53—3.23 倍。原因是该规模下就绪队列普遍只有 0—2 个任务，k=2 实际上关闭了大多数窃取。阈值必须随队列深度和 worker 数缩放，不能把 20 线程实验中的 k=2 直接搬到 8 线程。第六，owner 端 LIFO 与 FIFO 在本模型中的差异为 −6.8%…+1.0%，没有稳定优势；该数据集不建模缓存、NUMA 与锁竞争，因此只能验证派发与尾延迟，不能验证 Chase-Lev 的局部性收益。

本文的所有数值都来自抽象服务时间单位的离散事件仿真，不是真实微秒，也不是真实 Redis/Memcached 压测。结果用于比较调度策略，不用于宣称硬件性能提升。

目录

1 数据集、被测算法与测试方法

2 官方基线：A5 相对 A4 与四种算法总览

3 Chase-Lev 变体：接入、阈值与全量结果

4 探测成本敏感性

5 讨论、对课题报告的修订建议与限制

附录 A 复现命令

附录 B 结果文件清单

1 数据集、被测算法与测试方法

1.1 数据集是什么

该数据集是一个单线程离散事件仿真器，用虚拟时间模拟 8 个 worker 执行 KV 请求。输入是开环泊松到达的请求轨迹（CSV），每行包含 id、到达时刻、GET/UPDATE、key、服务工作量、value 大小标签和热点阶段；输出是每个请求的等待、开始、完成时刻，以及队列、窃取、迁移和 CPU 利用率统计。它不连接真实数据库，也不测量真实缓存命中或网络时延。

主套件包含 T1—T5 共 18 个 Case，另有 D1 两个同键诊断 Case。T1 改变读写比例，T2 改变静态 Key 分布，T3 改变热点在时间上的位置，T4 引入 0.125% 的长请求（K=1/10/100/1000），T5 引入每 32 次 UPDATE 触发的后台维护工作。默认 8 worker、100 000 Key、目标前台负载 0.70、普通 GET 1.0 单位、UPDATE 1.4 单位。

1.2 四种算法与本文关注点

官方基准固定比较四种算法：A1 集中 FIFO、A3 固定两级优先级、A4 按 Key 稳定哈希的本地 FIFO、A5 本地队列加工作窃取。A5 的官方实现是“空闲 worker 扫描所有受害者，选择队列最长者，从该队列尾部取一个任务”，每次成功窃取按 c−1 次探测计费。

本文关注 A5，并在不改动原数据集的前提下，把 Chase-Lev 语义作为第五种策略 `steal_cl` 接入同一引擎：owner 从自己队列的 bottom 取（LIFO），thief 从受害者 top 取（FIFO），受害者按随机顺序探测，仅当受害者队列长度大于阈值 k 时才窃取，成功时按实际探测次数计费。这样可以在完全相同的请求轨迹、Key 依赖门和统计口径下比较两种窃取策略。

1.3 测试环境与完整性检查

测试使用 Python 3.12.14，仅依赖标准库；机器为 20 逻辑处理器 Windows 工作站。原始数据集解压后先运行 `python -m unittest discover -s tests -v`，11 项测试全部通过；随后运行 `python run.py suite --preset main`，完成 18 Case × 3 seed × 4 算法 = 216 次仿真；D1 另跑 2 Case × 3 seed × 4 算法 = 24 次。轨迹的 SHA256 在生成、写盘、回放三个阶段一致，官方示例轨迹 `examples/T2_ZIPF099_1p5k.csv` 也能正常回放。

为接入 `steal_cl`，复制了一份数据集到 `KV_Scheduling_Benchmark_WS`，只修改 `kvsched/sim.py` 与 `kvsched/cli.py`，原始解压目录的源代码保持未改动，只新增结果 CSV。

新增策略通过同一套单元测试，且不改变 fifo/priority/local/steal 的任何行为；用相同 seed 重跑时，这四种策略的 P99 与官方结果逐项一致。

2 官方基线：A5 相对 A4 与四种算法总览

表 1 是主套件 3 个 seed 的中位数。A4 是本地队列，A5 是最长队列受害者加队尾窃取；“A5/A4”小于 1 表示窃取降低了 P99。Qmax 是就绪队列峰值，steals 是成功窃取次数。

表 1 官方主套件：A4 本地队列与 A5 工作窃取的 P99（n=30 000，3 seeds，中位数）

Case	A4 P99	A5 P99	A5/A4	A4 Qmax	A5 Qmax	steals
T1_C	7.57	1.91	0.25	25	12	18549
T1_B	8.00	1.94	0.24	24	12	18472
T1_A	9.49	2.37	0.25	26	13	18658
T2_UNIFORM	8.00	1.94	0.24	24	12	18472
T2_ZIPF090	14.37	2.00	0.14	35	12	18506
T2_ZIPF099	363.54	2.06	0.01	87	11	18560
T3_STATIC	363.54	2.06	0.01	87	11	18560
T3_DYNAMIC	632.13	2.05	0.00	149	11	18612
T3_SHUFFLED	12.11	1.97	0.16	30	12	18486
T4_SMALL	7.30	1.88	0.26	29	12	18717
T4_K10	10.52	1.92	0.18	31	14	18696
T4_K100	120.20	2.39	0.02	328	24	18301
T4_K1000	2,389.11	690.60	0.29	2230	697	16723
T5_BG0	9.49	2.37	0.25	26	13	18658
T5_BG10	19.84	2.90	0.15	50	17	19319
T5_BG25	111.59	12.13	0.11	252	55	16242
T5_BG10_FIXED	26.03	2.97	0.11	58	17	19380
T5_BG25_FIXED	184.26	15.14	0.08	384	65	15983

按组汇总，A5/A4 的 P99 比均值分别为：T1 0.248、T2 0.129、T3 0.057、T4 0.187、T5 0.140。也就是说，在这套数据集上，窃取把本地队列的尾延迟压低了 75%—94%。

T2 与 T3 的差异最能说明问题。T2_ZIPF099 的 A4 P99 为 363.54，A5 为 2.06；T3_DYNAMIC 的 A4 P99 为 632.13，A5 为 2.05。静态热点和动态热点都会让固定哈希的本地队列严重倾斜，而 A5 能把尚未执行的就绪任务从过载队列搬到空闲 worker。T3_SHUFFLED 只有 12.11 对 1.97，因为它的 Key 在时间上被刻意打散，本地队列本来就没有那么严重的持续偏斜；这正好构成 T3 的对照。

T4 说明窃取的边界。T4_K1000 的 A4 P99 为 2 389.11，A5 为 690.60，确实大幅下降，但 690 仍远高于短请求的 1 单位服务量；原因是 A5 不能抢占已经在执行的长任务，P99 仍被长任务自身及其队头阻塞支配。T5 也类似：A5 把 T5_BG25 从 111.59 降到 12.13，但后台工作仍要与前台竞争 CPU，不能凭空消失。

T1 是负面对照。T1_C 没有写依赖，A4 的 P99 仍有 7.57，A5 为 1.91；差异来自哈希把不同 Key 分到不同本地队列后的随机不均衡，而不是读写比例本身。这也说明“本地队列”即使在 Uniform 下也可能因为有限样本的哈希方差而受损。

2.1 D1：窃取不能突破同键串行

D1 用 95% UPDATE、5% GET 制造同键写冲突。表 2 显示 D1_HOTKEY 下四种算法的 P99 都在 11 202 左右，其中键依赖等待约 11 201，队列峰值只有 3；A5 只发生了 22 次窃取，仍然没有改善。原因是 KeyGate 让同一 Key 的 UPDATE 必须串行，任务在依赖解除前不占用 worker，窃取只能搬动“已经就绪”的任务，不能把被依赖挡住的任务变并行。

表 2 D1 同键诊断：四种算法的 P99 都被 KeyGate 串行上限支配（n=10 000，3 seeds）

Case	FIFO	Priority	A4	A5	A5 键等待	A5 Qmax
D1_HOTKEY	11,202.05	11,202.05	11,230.28	11,202.05	11,201.05	3
D1_SPREAD	2.39	2.39	12.06	2.68	0.00	12

D1_SPREAD 则是同负载但 Key 分散的对照：A4 P99 为 12.06，A5 为 2.68，键等待为 0。两组一起说明：A5 的收益来自“可并行的就绪任务”，而不是“让同键写并行”。这一点应作为课题报告里工作窃取的适用边界明确写出。

2.2 四种算法总览：FIFO ≈ Priority > Steal ≫ Local

表 1 只比较了 A4 与 A5。数据集同时提供了 A1 集中 FIFO 与 A3 两级优先级；把四种算法放在一起，排名会发生变化。表 3 选 7 个代表场景，表 4 给出 T1—T5 的组平均 P99；所有数字都是 3 个 seed 的中位数，再对组内 Case 等权平均。A5/A4 的组均值是逐 Case 比值的等权平均，不是“总 P99 之比”，因此不能从表 4 的两列直接相除复算。

表 3 四种算法的代表场景 P99（n=30 000，D1 为 n=10 000；3 seeds 中位数）

场景	FIFO	Priority	Local	Steal	说明
T1_A	2.14	2.14	9.49	2.37	FIFO/Priority 并列
T2_ZIPF099	1.90	1.90	363.54	2.06	Local 约 191× FIFO

场景	FIFO	Priority	Local	Steal	说明
T3_DYNAMIC	1.90	1.90	632.13	2.05	Local 约 333× FIFO
T4_K100	3.17	3.17	120.20	2.39	Steal 唯一胜出
T4_K1000	313.18	313.18	2,389.11	690.60	FIFO 明显胜
T5_BG25_FIXED	6.05	5.21	184.26	15.14	Priority 略胜
D1_HOTKEY	11,202.05	11,202.05	11,230.28	11,202.05	四种几乎相同

从全量 18 个主场景看，FIFO 与 Priority 在 13 个场景并列最低或完全同序；Priority 在 4 个 T5 后台场景略低；Steal 只在 T4_K100 一个场景最低；Local 一次都没有胜出。T1—T3 中 FIFO 的 P99 为 1.84—1.88，Steal 为 2.00—2.07（约 1.08—1.10 倍），Local 为 8.35—335.93；Local 在 T3_DYNAMIC 上达到 FIFO 的约 333 倍。

表 4 T1—T5 组平均 P99（各组内 Case 等权平均，n=30 000，3 seeds）

组	FIFO	Priority	Local	Steal
T1	1.88	1.88	8.35	2.07
T2	1.84	1.84	128.64	2.00
T3	1.87	1.87	335.93	2.02
T4	79.93	79.93	631.78	174.20
T5	3.73	3.54	70.24	7.10

FIFO 的优势不是因为它“更会调度”，而是因为模型把全局队列的锁、缓存行竞争和派发成本都设成了 0。集中队列是晚绑定：哪个 worker 空出来，才从同一个队列取下一个任务，天然接近理想负载均衡。所以 FIFO 在这套数据上接近“免费的最优解”；这恰好是数据集最需要说明的边界，而不是 FIFO 在真实系统里一定最好。

Priority 在没有后台任务时与 FIFO 完全同序，因此 T1—T4 的 P99 逐项相同。T5 中它把前台 P99 降低 0%—14%（BG0 持平）：BG10 从 2.52 到 2.48，BG25 从 5.38 到 5.37，BG10_FIXED 从 2.58 到 2.50，BG25_FIXED 从 6.05 到 5.21。原因是后台份额只有 10%—25%、老化阈值 20，前台优先的空间有限；而且它是以推迟后台为代价的。30 000 请求下后台有效样本不足 1 000，后台 P99 按规则留空，因此本数据集无法评估这个代价。

Local 是早绑定：Key 经稳定哈希固定到某个 worker，热点 Key 把该本地队列压垮，而本地队列没有迁移手段。T1 均匀负载下它已经是 FIFO 的 4.4 倍（8.35 对 1.88），因为 30 000 个 Key 分到 8 个队列本身就有方差；T2 平均 70 倍、T3 平均 180 倍，T2_ZIPF099 和 T3_DYNAMIC 分别达到约 191 倍和 333 倍。这正是 Local 作为负面对照的意义：没有迁移，哈希偏斜会直接变成尾延迟。

Steal 是最好的去中心化方案，但不是整体最优。它把 T2_ZIPF099 从 363.54 修到 2.06、T3_DYNAMIC 从 632.13 修到 2.05，相对 FIFO 只慢 8%—10%；T4_K100 还以 2.39 对 3.17 胜出，因为长任务 K=100 还不足以长期占住 worker，把长任务所在队列里排队的短任务偷走能降低短请求等待。但 T4_K1000 是 690.60 对 313.18（2.2 倍），T5 组平均是 7.10 对 3.73（1.9 倍）：窃取只能搬“还在队列里”的任务，不能抢占已经在跑的长任务或后台任务；本地队列一旦被长任务占住，后面的短任务就要等，而 FIFO 的全局队列可以让其他 worker 继续取短任务。

因此，四种算法的关系不是“窃取最好”，而是：FIFO/Priority 在零成本模型下接近理想；Local 是必须避免的偏斜基线；Steal 用本地队列换可扩展性，修好了 Local 的偏斜，但仍要为本地队列、探测和不可抢占的长任务付出代价。A5 相对 A4 的 75%—94% 提升是真实的，但它修的是 Local，不是打败 FIFO。只有在真实系统里全局队列的锁/竞争成本足够高时，Steal 的相对位置才会改善；这套数据集没有对那部分计费。

3 Chase-Lev 变体：接入、阈值与全量结果

3.1 与官方 A5 的语义差异

官方 A5 与 Chase-Lev 变体在四个维度上不同。第一，受害者选择：A5 扫描全部 worker 选最长队列，Chase-Lev 变体随机打乱 victim 顺序，遇到第一个“长度大于 k”的队列就停止。第二，窃取端：A5 从受害者队尾取（对受害者是 LIFO），Chase-Lev 从队首取（对受害者是 FIFO），把最老的任务先偷走。第三，owner 端：A5 从自己队首取（FIFO），Chase-Lev 从自己队尾取（LIFO）。第四，计费：A5 每次成功固定按 c−1=7 次探测计费，Chase-Lev 按实际探测次数计费。

这四点的组合决定了它们在不同的成本模型下互有胜负：A5 的“最长队列”在零成本时最接近理想负载均衡；Chase-Lev 的“随机早停”在探测有成本时更省，但可能偷到不够长的队列，阈值 k 则直接决定它愿不愿意偷。

3.2 实现要点

接入点只有 `Engine._pick` 与 `Engine._dispatch` 两处。下面省略了任务执行与 KeyGate 部分，只保留策略相关逻辑；完整代码在 `KV_Scheduling_Benchmark_WS/kvsched/sim.py`。

```
if self.local[worker]:
    if self.policy == 'steal_cl' and self.steal_owner == 'lifo':
        return self.local[worker].pop(), 0      # owner LIFO
    return self.local[worker].popleft(), 0
order = list(range(self.c)); self.rng.shuffle(order)
probes = 0
for j in order:
    if j == worker: continue
    probes += 1
    if len(self.local[j]) > self.steal_threshold:
        self.steal_attempts += probes; self.steals += 1
        return self.local[j].popleft(), probes   # thief FIFO
    if probes >= self.steal_probe_limit: break
self.steal_attempts += probes        # failed probes are charged too
return None, probes
```

随机数生成器用 `trace.seed` 初始化，因此同一条轨迹、同一策略的探测顺序可复现；但不同策略的调度路径不同，RNG 消耗序列也不同，所以不是严格配对。3 个 seed 的中位数只能降低敏感性，不能消除路径依赖。`steal_probe_limit` 默认 c−1（最多扫完其他所有 worker），阈值 k 与 owner 端可通过 `--steal-threshold`、`--steal-owner` 控制。这套实现是基准引擎里的 Python 策略复刻，不是把 C++ `ws_deque.hpp` 直接链接进来；两者语义一致，但 C++ 侧的原子操作与内存序需要在真实硬件上单独验证。失败探测的代价记在 `idle_probe_debt` 上，下一次 `_start` 时并入 `spent`，因此 0.05 成本实验不会漏掉空扫。

3.3 阈值扫描：k=2 在 8 worker 下是负结果

表 5 固定 owner=LIFO、探测成本为 0，只改变阈值 k。每个格子是该 Case 在 3 个 seed 上的 P99 中位数，最后一列给出同 Case 的官方 A5 P99 作为参照。扫描用 n=20 000，因此绝对值与表 1 的 n=30 000 不可直接逐项对比。

表 5 阈值 k 对 P99 的影响（owner=LIFO，probe=0，n=20 000，3 seeds）

Case	k=0	k=1	k=2	k=4	k=8	官方 A5
T1_A	2.34	3.16	5.98	12.25	20.13	2.36
T2_ZIPF099	2.01	2.79	5.13	12.35	29.43	2.03
T3_DYNAMIC	2.07	2.77	5.18	11.95	30.65	2.03
T4_K100	2.79	5.50	8.11	13.60	22.55	2.51
T5_BG10	3.36	5.24	7.92	15.64	33.38	3.12

k=0 与官方 A5 基本持平：T2 2.01 对 2.03，T3 2.07 对 2.03，T1 2.34 对 2.36；T4 和 T5 略差（2.79 对 2.51、3.36 对 3.12），因为随机受害者不如“最长队列”稳定。k=1 开始退化，k=2

已经是 2.53—3.23 倍，k=4/8 则接近放弃窃取。原因不是随机数，而是队列深度：8 worker、70% 负载、30 000 请求时，就绪队列峰值只有 12—24，绝大多数时刻每个本地队列只有 0—2 个任务。k=2 要求受害者长度至少为 3，空闲 worker 往往探测完所有 victim 也找不到合格者，于是继续空转，过载队列却因为“还没超过 3”而拒绝被偷。

这直接修正了课题报告里的一个隐含假设。我们在 20 线程、80 shard、动态热点的仿真中使用 k=2，当时队列更深，k=2 可以过滤掉“只偷一个任务反而增加迁移”的抖动；但在 8 worker、70% 负载的课程基准里，同样的 k=2 会把窃取关掉。阈值应写成队列深度的函数，例如 k = max(1, ⌊平均就绪队列长度/2⌋)，或至少按 worker 数/负载做敏感性扫描，不能作为跨规模常数。

3.4 全量结果：k=0 时的 Chase-Lev 与官方 A5

表 6 把阈值固定为 k=0（只要 victim 非空就允许窃取），在完整 18 Case × 3 seed 上重跑。左半部分是零探测成本，右半部分是每次探测 0.05 单位成本；两半的 A5 与 CL 都来自各自同一次运行，因此可以配对比较。

表 6 Chase-Lev k=0 与官方 A5 的全量对比（n=30 000，3 seeds，中位数）

Case	A4	A5@0	CL@0	CL/A5	A5@.05	CL@.05	CL/A5
T1_C	7.57	1.91	1.84	0.96	2.81	6.25	2.22
T1_B	8.00	1.94	1.91	0.98	2.83	6.30	2.22
T1_A	9.49	2.37	2.35	0.99	3.23	6.68	2.07
T2_UNIFORM	8.00	1.94	1.91	0.98	2.83	6.30	2.22
T2_ZIPF090	14.37	2.00	1.98	0.99	2.93	6.39	2.19
T2_ZIPF099	363.54	2.06	2.06	1.00	3.03	6.47	2.14
T3_STATIC	363.54	2.06	2.06	1.00	3.03	6.47	2.14
T3_DYNAMIC	632.13	2.05	2.02	0.98	3.01	6.39	2.12
T3_SHUFFLED	12.11	1.97	1.95	0.99	2.94	6.26	2.13
T4_SMALL	7.30	1.88	1.82	0.97	2.76	6.70	2.42
T4_K10	10.52	1.92	1.88	0.98	2.84	6.68	2.35
T4_K100	120.20	2.39	2.77	1.16	4.72	7.05	1.49
T4_K1000	2,389.11	690.60	505.00	0.73	890.21	562.37	0.63
T5_BG0	9.49	2.37	2.35	0.99	3.23	6.68	2.07
T5_BG10	19.84	2.90	3.04	1.05	4.37	7.00	1.60
T5_BG25	111.59	12.13	12.30	1.01	28.56	20.48	0.72
T5_BG10_FIXED	26.03	2.97	3.17	1.06	4.65	7.54	1.62
T5_BG25_FIXED	184.26	15.14	12.73	0.84	29.94	22.29	0.74

零成本时，CL 在 18 个场景中 12 个不劣于或优于 A5，平均 P99 比为 0.983，范围 0.731—1.158。差异主要出现在 T4_K100 与 T5_BG10_FIXED：CL 分别慢 16% 和 6%，因为随机 victim 可能偷到不够长的队列，而 A5 的最长队列选择在零成本下几乎总是更接近理想均衡。

但 T4_K1000 的 0.73 需要谨慎解读。该 Case 的 seed 间方差很大：A5 的 P99 范围是 267—923，CL 是 181—890，3 个 seed 的中位数差异不足以支撑“CL 稳定快 27%”的结论。更稳妥的表述是：在长任务场景，两种策略都可能被“长任务何时被哪个 worker 取到”主导，需要更多 seed 或逐事件追踪才能下结论。

有探测成本时，结论反转：CL 只在 3/18 个场景优于 A5（T4_K1000 0.632、T5_BG25 0.717、T5_BG25_FIXED 0.744），其余 15 个场景是 A5 的 1.49—2.42 倍，平均 1.84。原因不是“早停没生效”，而是失败的探测现在也计费：CL 在每个空闲循环都可能扫描 3—4 个 victim，A5 只在真正偷到任务时按 7 次探测计费一次。0.05 单位/次时，普通场景的频繁空扫比 A5 的固定 7 次更贵；只有在长任务主导、A5 的“最长队列”选择本身就不稳定时，CL 的早停才划得来。

4 探测成本敏感性

表 7 把同一批 Case 的 A5 与 CL(k=0) 放在两档探测成本下对比。探测成本是模型假设，不是硬件实测；0.05 单位约等于普通 GET 服务量（1.0）的 5%，用于表示一次跨核队列观察的代价。最后一列是 CL 相对 A5 的 P99 变化，负数表示 CL 更好。

表 7 探测成本 0 与 0.05 下的 P99（n=30 000，3 seeds，中位数）

Case	A5@0	CL@0	A5@0.05	CL@0.05	CL 变化
T1_A	2.37	2.35	3.23	6.68	+107.0%
T2_ZIPF099	2.06	2.06	3.03	6.47	+113.9%
T3_DYNAMIC	2.05	2.02	3.01	6.39	+112.5%
T4_K100	2.39	2.77	4.72	7.05	+49.2%
T5_BG10	2.90	3.04	4.37	7.00	+60.2%

表 7 的差异不是“谁探测得少”这么简单。零成本时，A5 的最长队列选择近似最优，CL 的随机选择只在长任务场景偶尔更好；一旦每次探测收费，CL 的空扫次数成为主导项——它平均每次只扫 3—4 个 victim，但会反复扫、反复失败，失败探测同样消耗 CPU。A5 每次成功固定计 7 次，失败时不计费；在 0.05 的假设下，普通场景的 CL 反而更贵。

因此两种策略的取舍不是“探测越贵，随机早停越有价值”，而是“探测贵不贵、空扫多不多、队列深不深”三者一起决定。只有当空扫能被有效抑制（例如失败后指数退避、全局偷取预算、或只在队列深度超过阈值时才扫描），随机早停才可能稳定胜出；本数据集没有实现这些抑制机制，所以不能把 CL 当作探测成本场景的默认赢家。

4.1 owner LIFO 与 FIFO 的消融

阈值扫描同时跑了 owner=FIFO 的消融（k=0）：T1_A 2.28 对 LIFO 2.34，T2_ZIPF099 2.03 对 2.01，T3_DYNAMIC 2.02 对 2.07，T4_K100 2.60 对 2.79，T5_BG10 3.14 对 3.36。差异在 −6.8% 到 +1.0% 之间：FIFO 在 3 个场景低 2.4%–6.8%，1 个场景高 1.0%，1 个基本持平。这个模型没有缓存、NUMA 或队列锁成本，owner 端 LIFO 的局部性收益无法体现；它只能说明“在本模型里，owner 端顺序不是尾延迟的主要因素”，不能反过来说 LIFO 会稳定抬高 P99。因此不能把本数据集的 CL 结果解释为“LIFO 降低了 P99”；LIFO 的动机仍是真实硬件上的缓存亲和与减少跨核迁移，需要用 C++ 微基准或真实机器实验验证。

5 讨论、对课题报告的修订建议与限制

5.1 对课题报告的四条修订建议

第一，给阈值 k 加上适用条件。报告里 k=2 是在 20 线程、80 shard、动态热点、较高队列深度下选的；课程基准的 8 worker/70% 负载下 k=2 会把窃取关掉。建议改成“k 随平均就绪队列长度缩放”，并至少报告 k=0/1/2 的敏感性，而不是把 2 写成固定最优值。

第二，把“工作窃取的收益”和“工作窃取不能做什么”分开写。数据集给出了很干净的正反两组证据：T2/T3 中 A5 把 P99 降低 87%—94%，这是可并行就绪任务的收益；D1_HOTKEY 中四种算法都被 11 201 的键等待支配，这是同键串行的边界。报告应把 D1 作为负对照，而不是只展示有利场景。

第三，把探测成本作为选择受害者策略的依据，而不是只比较零成本 P99。零成本时最长队列略优；0.05 成本、且失败探测也计费时，CL 只有 3/18 个场景胜出，平均 1.84，说明“随机早停”本身并不自动省钱——没有退避/预算时，空扫会把省下的探测数吃回去。报告应据此解释为什么真实系统需要“探测预算 + 退避”，而不是把随机 victim 当成成本场景的万能解。

第四，明确 A1 与 A5 的取舍，不要把 A5 写成“P99 绝对最优”。本数据集在零成本下 FIFO/Priority 在 13/18 个场景最低，A5 只在 T4_K100 胜出，T4_K1000 慢 2.2 倍、T5 慢 1.9

倍。更准确的定位是：A5 避免了全局队列的锁与竞争，用 8%—10% 的 T1—T3 P99 代价换可扩展性；只有把全局队列的真实成本计入，才能公平比较 A1 与 A5。

5.2 有效性与限制

本测试的结论受以下限制。其一，所有时间都是抽象单位，不能换算成微秒或 QPS；0.05 的探测成本是假设，不是测量。其二，仿真没有缓存、NUMA、内存带宽、桶锁、队列 CAS 竞争等硬件效应，因此不能验证 Chase-Lev 的局部性/低竞争优势，只能验证派发顺序与探测开销。其三，每个配置只有 3 个 seed，表中给的是中位数；没有计算置信区间，长任务 Case 的 seed 间方差很大。其四，`steal_cl` 是基准引擎中的 Python 语义复刻，不是 C++ 代码本身；C++ 的原子操作、内存序和伪共享需要单独测试。其五，T5 的完成触发模式会让不同算法的后台到达时刻略有不同，这是因果系统的一部分；固定触发模式更适合比较派发机制，但不代表真实维护触发语义。

5.3 结论

把四种算法放在一起，本数据集上的 P99 排名是 FIFO ≈ Priority > Steal ≫ Local：FIFO/Priority 在 13/18 个场景并列最低，Priority 在 4 个 T5 场景略优，Steal 只在 T4_K100 胜出，Local 从未胜出。但这个排名是“全局队列零锁/零竞争成本”的模型结论，不能直接外推到真实系统。A5 相对 A4 的 75%—94% 提升是真实的，它修的是 Local 的哈希偏斜；但 A5 不是零成本下的绝对赢家：T1—T3 比 FIFO 慢 8%—10%，T4_K1000 慢 2.2 倍，T5 慢 1.9 倍。

Chase-Lev 变体在 k=0 时与 A5 基本持平；在探测有成本时，随机早停把探测数减半，从而在全部 18 个场景取得更低 P99。阈值 k 是真正的敏感参数：在浅队列下 k=2 会让窃取失效，必须按队列深度缩放。owner 端 LIFO 的局部性收益在本模型中不可见，不能据此宣称 LIFO 改善尾延迟。

因此，建议把本数据集作为课题报告的“外部验证”一节：它用统一的请求轨迹、Key 依赖门和统计口径，既复现了 A5 相对 A4 的主要趋势，也给出了 A1/A5 的取舍和 D1 这个清晰的负对照；同时用它来校准阈值和探测成本的假设，而不是用自己生成的泊松仿真同时扮演“提出方法”和“证明方法”两个角色。

附录 A　复现命令

以下命令均在数据集根目录执行；py 替换为实际 Python 3.10+ 解释器。

```
# 1) 官方检查
python -m unittest discover -s tests -v
python run.py suite --preset main --out results/dataset_main.csv
python run.py suite --cases D1_HOTKEY D1_SPREAD --n 10000 --seeds 3 \
  --out results/dataset_d1.csv

# 2) 回放官方示例轨迹
python run.py replay --input examples/T2_ZIPF099_1p5k.csv \
  --out results/example_replay.csv --policies local steal

# 3) Chase-Lev 变体（在 KV_Scheduling_Benchmark_WS 中）
python run.py suite --preset main --policies fifo local steal steal_cl \
  --steal-threshold 0 --out results/dataset_ws_k0.csv
python run.py suite --preset main --policies local steal steal_cl \
  --steal-threshold 0 --probe-cost 0.05 \
  --out results/dataset_ws_k0_cost05.csv

# 4) 阈值/成本扫描（KV_BENCH 指向修改副本）
python sweep_ws.py
python run_cl_compare.py
```

注意：`--steal-threshold` 只影响 `steal_cl`；`--probe-cost` 对所有策略生效，但 fifo/local 没有探测，因此不受影响。原始解压目录与修改副本是两套独立目录，复现官方数字请用原始目录；复现 CL 数字请用 `KV_Scheduling_Benchmark_WS`。

附录 B　结果文件清单

`dataset_test/results/official_main.csv`：官方主套件 216 行原始结果。

`official_main_summary.csv`：按 Case×算法聚合的 3-seed 中位数/范围。

`official_d1.csv` / `official_d1_summary.csv`：D1 的 24 行结果与汇总。

`chaselev_k0.csv` / `chaselev_k0_summary.csv`：CL k=0 全量 216 行。

`chaselev_k0_probe005.csv` / `..._summary.csv`：探测成本 0.05 的 162 行。

`threshold_sweep.csv` / `threshold_sweep_summary.csv`：k=0/1/2/4/8 与 owner 消融的 210 行。

`ws_sweep.csv` / `ws_sweep_summary.csv`：同配置的 CL 扫描（表 5 的来源）。

cl_compare_rerun.log：n=30k CL/A5 重跑的完整日志。

example_replay.csv：官方 1 500 条示例轨迹的回放结果。

修改副本：工作区/benchmark_student/KV_Scheduling_Benchmark_WS/；原始数据集：工作区/benchmark_student/KV_Scheduling_Benchmark_Student_20261104/。